

TGNN: Enhancing Pixel Tracking Detection via LLM-driven Annotation and GAT-powered Structural Representation

Shenping Xiong*
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
xiongshenping@iie.ac.cn

Xutong Wang
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
wangxutong@iie.ac.cn

Ze Jin*
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
jinze@iie.ac.cn

Xinyu Liu†
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
liuxinyu@iie.ac.cn

Haoqiang Wang*
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
wanghaoqiang@iie.ac.cn

Zhen Chen*
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
chenzhen@iie.ac.cn

Ru Tan*
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
tanru@iie.ac.cn

Qixu Liu*†
Institute of Information Engineering,
Chinese Academy of Sciences
Beijing, China
liuqixu@iie.ac.cn

Abstract

Web tracking is increasingly pervasive, raising serious concerns about user privacy and security. Among existing techniques, pixel tracking is particularly stealthy and cost-effective, embedding invisible images that exfiltrate user activities to third-party servers. Current defenses, including filter list blocking and conventional machine learning, often fail to capture the cross-site associations that enable pixel tracking to evade detection.

To address this limitation, we introduce TGNN, a framework that formulates pixel tracking detection as an edge classification task on a Tracking Directed Graph (TDG), which models third-party associations across websites. TGNN encodes HTTP traffic into structured quadruples and learns both semantic features and interaction patterns. To overcome the scarcity of reliable labels, we propose a large language model (LLM)-based annotation method that leverages minimal expert supervision to produce high-quality labels, significantly improving detection. Experiments conducted on traffic from the Alexa top-10K websites demonstrate that TGNN substantially outperforms existing baselines, while the LLM-based annotation achieves accuracy comparable to expert curation. Our large-scale measurement reveals that at least 16.74% of websites engage in pixel tracking via major third-party infrastructures, establishing cross-domain tracking as a pervasive practice in the

wild and indicating a potential privacy threat in the modern Web ecosystem.

CCS Concepts

• Security and privacy → Social network security and privacy.

Keywords

web tracking, privacy, graph neural networks, large language model

ACM Reference Format:

Shenping Xiong, Xutong Wang, Ze Jin, Xinyu Liu, Haoqiang Wang, Zhen Chen, Ru Tan, and Qixu Liu. 2026. TGNN: Enhancing Pixel Tracking Detection via LLM-driven Annotation and GAT-powered Structural Representation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792149>

1 Introduction

Web tracking technologies have become pervasive, exerting profound implications on user privacy and security [10]. Tracking user behavior is now a widespread yet highly contentious practice within the online ecosystem. Studies show that more than 80% of major global websites incorporate third-party tracking technologies [9]. Among these, pixel tracking has emerged as a particularly prevalent technique, in which pixel-level images or code snippets are embedded into webpages to surreptitiously collect user activity data [36]. Such data can include browsing histories, click patterns, and even personal information, often gathered without users' explicit awareness or consent [6]. The privacy risks associated with such practices have attracted increasing public concern, motivating governments worldwide to introduce regulations aimed at protecting personal data [2, 37].

Although web tracking itself is generally lawful, issues such as excessive data collection, breaches, and illicit data trading pose

*Also affiliated with School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China.

†Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792149>

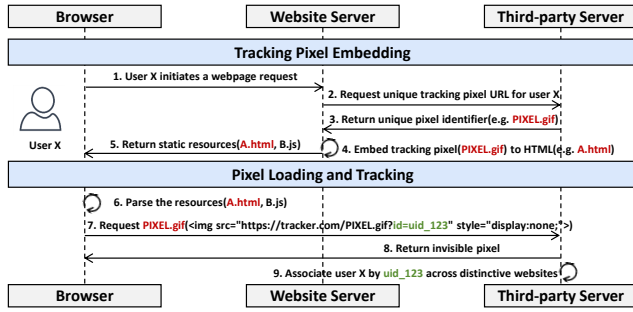


Figure 1: The workflow of pixel tracking.

significant threats to users [11, 20, 28, 40]. Merely relying on regulatory efforts [12, 13, 16] to restrict or mitigate pixel tracking is highly challenging, given its inherently stealthy and technically sophisticated nature [36]. Existing countermeasures, including cookie blocking and fingerprinting defenses [4, 7], remain inadequate for effectively mitigating pixel-based tracking.

Nevertheless, detecting pixel tracking remains challenging due to its **stealthy and adaptive nature**. Trackers often mimic benign web resources through disguised network requests, evading rule-based detection tools like ad blockers and resulting in high false negative rates [29, 35]. Moreover, these trackers employ evasion techniques such as dynamic URL parameterization [14], which substantially undermine the effectiveness of static analysis approaches, including tools such as JStap [17]. A further persistent challenge is **the scarcity of reliable ground-truth data** for machine learning.

Current approaches largely depend on crowd-sourced filter lists [5, 24, 39, 50], which are inherently incomplete and fail to identify trackers outside their predefined domains [50]. While expert labeling can ensure quality, the manual process is costly and inefficient, making it infeasible to generate the large-scale, high-quality datasets required for robust machine learning models.

Conventional tracking detection methods predominantly rely on rule-based filtering [21, 42, 44, 45] or supervised machine learning [3, 19, 24, 30, 34, 38, 39, 41]. These approaches, however, face two fundamental limitations in detecting modern pixel tracking. First, their effectiveness is heavily contingent on manual feature engineering and static rules, which lack the adaptability to counter evolving evasion tactics. Second, and more critically, these methods are primarily designed for single-domain analysis—such as string matching against domain lists or scrutinizing JavaScript execution within a page. This inherent design limits their capacity to model and identify the cross-site data exchange pathways that constitute the core mechanism of pixel tracking.

To this end, we propose a novel approach TGNN that leverages graph neural networks empowered by large language models to enhance the detection of pixel tracking. We begin by encoding HTTP traffic into structured quadruples to capture semantic features from requests and responses. To represent the pervasive data collection conducted by third-party trackers across unrelated websites, we model the network traffic as a tracking directed graph (TDG). In this graph, nodes represent websites while edges represent requests or responses from a source site to a destination. TDG effectively characterizes the interactive relationships between first-party websites

and third-party tracking entities. To tackle the critical challenge of reliable ground-truth generation, TGNN employs tailored prompt engineering to guide the LLM [46] in analyzing network requests, effectively bridging the gap between small-scale expert labels and the large-scale high-quality datasets required for robust model training. We leverage this partial LLM-annotated TDG to train a Graph Attention Network (GAT) classifier. The trained model is subsequently applied to infer and detect pixel tracking among the unlabeled edges within the graph. By evaluating TGNN on a dataset collected from the Alexa top-10K websites, a portion of which was manually annotated by domain experts as ground truth, we demonstrate that TGNN achieves superior performance in tracking detection. Our framework achieves an F_1 -Score of 92.24% in detecting connected requests and 84.49% for isolated requests, outperforming state-of-the-art graph-based tracking detection methods. Furthermore, the LLM-driven annotation method attains an F_1 -Score of 98.19% on the ground-truth set, a substantial improvement over the 55.14% achieved by rule-based methods, underscoring the critical role of LLMs in generating large-scale, high-quality labels. Furthermore, our TGNN framework revealed several concerning findings in real-world web environments. After classifying the analyzed websites, we observed that approximately 50% of the traffic within the *arts*, *recreation*, and *sports* categories involved pixel tracking activities. Even more troubling, 46.9% of the traffic from websites targeting *kids & teens* exhibited pixel tracking behavior, posing a serious threat to minors' privacy and safety and demanding greater public attention. Through this work, we aim to raise awareness of the pervasive issue of pixel tracking and to promote stronger privacy protection measures. Our main contributions are as follows:

- To the best of our knowledge, we are the first to leverage LLMs for automated large-scale annotation in pixel tracking detection. Experimental results on a meticulously curated ground-truth dataset demonstrate that our LLM-based labeling method generates high-quality training labels, which substantially improves the performance of downstream detection models.
- We propose TGNN¹, a novel graph-learning framework for pixel tracking detection. TGNN constructs relational quadruples from raw HTTP traffic and employs Graph Attention Networks (GAT) to jointly learn semantic representations and cross-site interaction patterns, significantly improving the detection of stealthy pixel-based tracking.
- We conduct a systematic analysis of pixel tracking on the Alexa top-10K websites, quantifying its prevalence and revealing that trackers are heavily concentrated in news, entertainment, and e-commerce sectors. Our findings demonstrate that at least 16.74% of distinct domains interact with third-party tracking services, providing empirical evidence of the pervasive and cross-domain nature of modern web tracking.

2 Background

2.1 How to Track with Pixels?

Invisible tracking refers to the practice of monitoring user behavior on websites without their explicit knowledge or consent. This is often achieved through various techniques.

¹We released our code at <https://gitlab.com/WWW26TGNN/WWW26TGNN>.

Tracking pixels, typically implemented as 1×1 pixel images (also known as web beacons), allow websites to monitor user interactions, including page visits and email openings, without providing any visible indication to the user. Alternatively, through the use of parameterized URLs that embed unique identifiers in query strings, sites can trace user navigation across multiple pages and sessions, effectively linking discrete interactions into a coherent behavioral profile.

In recent studies on pixel tracking, Bekos et al. [6] highlighted that Facebook introduced the FBCLID tagging mechanism, which appends a one-time identifier whenever users click on external links from the Facebook platform. Although this identifier alone is not sufficient for continuous tracking, when it is combined with Facebook Pixel [6], a JavaScript-based tracking tool, it facilitates persistent monitoring of users' browsing activities across multiple websites, interactions, and time periods. This technique has also been employed in the context of political competition. Agarwal et al. [1] reported that it is extensively used on the websites of Indian political parties to track potential users, particularly on those operated by left-leaning parties.

Pixel tracking operates by embedding invisible resources into web pages, as illustrated in Figure 1 which details the workflow across the browser, website server, and third-party server. Upon receiving a page request, the website server embeds a unique tracking pixel URL (https://tracker.com/PIXEL.gif?id=uid_123) into the HTML content before delivering static resources to the browser. The URL is generated by the third-party server and containing a unique user identifier (`id=uid_123`). When the browser parses the HTML and loads the page, it automatically requests the pixel from the third-party server, which in response returns the pixel image along with a Set-Cookie header to store a cookie (`id=uid_123`) for its domain. This cookie is then automatically included in all subsequent requests to that third-party server, enabling the collection of user behavior data and the association of transmitted signals with specific user profiles across sessions and websites, thereby constructing a persistent behavioral history for cross-site user monitoring.

2.2 How to Evade Traditional Detection?

The technical structure of pixel tracking allows it to inherently bypass detection methods reliant on JavaScript auditing and domain-based filtering.

Conventional detection based on JavaScript behavior analysis targets the client-side script execution during resource parsing (Step 6 in Figure 1). Pixel tracking, in contrast, bypasses such analysis by operating at a different layer: it uses the standard HTTP protocol to automatically and passively transmit identifiers like cookies, User-Agent, and IP address, without relying on any JavaScript logic.

Furthermore, domain-based filtering detection primarily targets the request phase for the third-party pixel resource (Step 7 in Figure 1). However, the effectiveness of such filtering hinges on pre-defined blocklists (e.g., EasyList [44]), which can be systematically evaded through domain rotation and dynamic generation techniques—including subdomain brute-forcing and domain generation algorithms (DGA).

Additionally, email serves as a prevalent vector for pixel tracking. In marketing emails, senders often embed an invisible tracking pixel within the HTML content. Upon opening the email, the client automatically requests the pixel from a third-party tracking server. By appending a unique identifier to the pixel URL, the tracker can recognize individual recipients and record email open events. Unlike web-based tracking, this method operates without involvement from a website server; the tracking pixel is directly embedded into the email content. A key advantage of this approach is its simplicity and reliability, as it functions even in environments with strict JavaScript restrictions. However, a significant limitation lies in its inability to capture fine-grained user interactions, such as click events or subsequent browsing behaviors, beyond the initial opening event.

3 Problem Formalization

To accurately define the research problem in this paper, we define a **Tracking Directed Graph (TDG)** as $G = (V, E, X^V, X^E, \mathcal{T})$

where V is the set of nodes. In the experimental environment of this paper, the node set V consists of the traversal set and all the sites linked from the traversal set, which can be expressed as $V = U \cup \bigcup_{k=1}^{\infty} \{v \mid \exists u \in V_{k-1} : v \in \text{Links}(u) \setminus V_{k-1}\}$ where U is the set of websites in the traversal set, $V_0 = U$, and $\text{Links}(u)$ represents the set of websites linked from website u . V_k is the set of nodes obtained after k iterations of link traversal.

$E \subseteq V \times V$ is the set of directed edges, where an edge $e_{(u,v,i)} \in E$ indicates that there is a request from node u to node v during the i -th visit to node u . Each edge is associated with a feature vector $X^E \in \mathbb{R}^{|E| \times d_e}$, where d_e is the dimension of the edge features. Each node is also associated with a feature vector $X^V \in \mathbb{R}^{|V| \times d_v}$, where d_v is the dimension of the node features. $\mathcal{T} : E \rightarrow \{0, 1\}$ is a mapping function that assigns a label to each edge, where $\mathcal{T}(e) = 1$ indicates that edge e is a tracking request, and $\mathcal{T}(e) = 0$ indicates that it is not. Figure 7 in Appendix D illustrates an example of a TDG constructed from web traffic. Each node in the graph represents a website, and each directed edge represents a request from one website to another. Due to the characteristics of web traffic, multiple edges may exist between the same source and target nodes, each corresponding to a different request. To uniquely identify each edge, we denote it as a triplet $e_{(u,v,i)}$. Self-loop edges, where a node issues a request to itself (i.e., $u = v$), are also permitted in this setting.

Data Preprocessing. We consider a quadruple $C = (H_{\text{Request}}, H_{\text{Response}}, R_{\text{Request}}, R_{\text{Response}})$, where H_{Request} and H_{Response} denote the request header and response header, respectively, and R_{Request} and R_{Response} denote the request origin and response origin, respectively. The objective of the LLM-based labeling component introduced in § 4.1 is to model this quadruple, extract contextual information, and infer its tracking attribute $\mathcal{T}$. Specifically, the input to the LLM-based labeling component is the quadruple $(H_{\text{Request}}, H_{\text{Response}}, R_{\text{Request}}, R_{\text{Response}})$, and the output is a tracking attribute $\mathcal{T} \in \{0, 1\}$, where $\mathcal{T}$ is defined as the Boolean function. The goal of the LLM-based labeling component is to learn a model $M_\lambda : C \rightarrow \{0, 1\}$ that determines whether a given context C corresponds to tracking traffic.

To further evaluate the effectiveness of the model M_λ , we construct a small ground truth dataset $D_{\text{truth}} = \{(c_i, y_i)\}_{i=1}^N$, where

c_i denotes the i -th tracking assertion and $y_i \in \{0, 1\}$ represents its corresponding ground truth label. We define the model-expert F_1 -score metric as $\text{Consistency}(M_\lambda, D_{\text{truth}}) = (2\text{PR}) / (P + R)$, where P and R are defined as in Equation 1. Here, $\mathbb{1}(\cdot)$ denotes the indicator function, which takes the value 1 if the condition inside the parentheses is true, and 0 otherwise. This metric measures the consistency between the predictions of M_λ and the expert annotations.

$$P = \frac{\sum_{i=1}^N \mathbb{1}(M_\lambda(c_i) = 1 \wedge y_i = 1)}{\sum_{i=1}^N \mathbb{1}(M_\lambda(c_i) = 1)} \quad (1)$$

$$R = \frac{\sum_{i=1}^N \mathbb{1}(M_\lambda(c_i) = 1 \wedge y_i = 1)}{\sum_{i=1}^N \mathbb{1}(y_i = 1)}$$

In this work, we design a zero-shot prompting strategy to heuristically guide the large language model M_λ in data labeling. Specifically, we construct a prompt template that incorporates both task descriptions and representative features of positive and negative samples. The generated prompts are then fed into M_λ to obtain the prediction results $\hat{y}_i = M_\lambda(c_i)$.

Feature Extraction. To combine natural language information with graph structural information, each node and edge needs to be represented as a feature vector. We use an embedding model M_ω to map node v and edge e to d_v -dimensional and d_e -dimensional vector spaces, respectively, i.e., $M_\omega(v) = x_v \in \mathbb{R}^{d_v}$ and $M_\omega(e) = x_e \in \mathbb{R}^{d_e}$. M_ω aims to encode the natural language information of nodes and edges into vector representations, so it needs to ensure that semantically similar nodes and edges are mapped to similar vector representations. Specifically, the feature vector of a node v , denoted as x_v , is obtained by embedding the domain name information of node v into a vector space. Similarly, the feature vector of an edge e , denoted as x_e , is derived by embedding the destination URL, source domain, request header, and response header of edge e into a vector space. Formally, we define $x_e = M_\omega(H_{\text{Request}}) \| M_\omega(H_{\text{Response}}) \| M_\omega(R_{\text{Request}}) \| M_\omega(R_{\text{Response}})$, where each component is defined as described above.

TDG Edge Representation Learning and Classification. Based on the formal definitions above, our objective is to classify each edge in the TDG graph and determine whether it corresponds to a tracking request. Given a TDG graph $G = (V, E, X^V, X^E, \mathcal{T})$, we aim to learn an edge classification model f_θ that predicts the label $\hat{y}_e \in \{0, 1\}$ of an edge $e_{(u,v,i)}$ by leveraging its feature vector x_e , the feature vectors of its source node x_u and target node x_v , as well as the structural information of the graph. Specifically, the edge classification model is defined as $f_\theta(x_e, x_u, x_v, G) = \hat{y}_e$ where $\hat{y}_e = 1$ indicates that edge $e_{(u,v,i)}$ is predicted as a tracking request, and $\hat{y}_e = 0$ indicates that it is predicted as a non-tracking request. By training the edge classification model f_θ , we aim to accurately identify tracking requests within the TDG graph, thereby enabling effective detection and analysis of web tracking behaviors.

4 Methodology

4.1 Model Architecture

To address the formulated problem, we design a model architecture that tackles three key challenges.

• **Challenge 1: Effectively capturing the semantic information of nodes and edges.** The nodes and edges in the TDG graph contain rich semantic information, such as website content, request

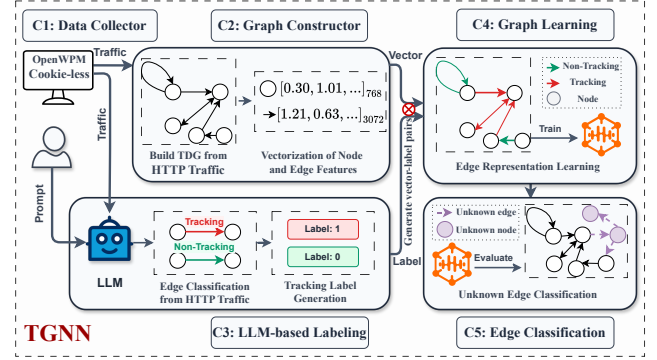


Figure 2: The overall architecture of TGNN.

headers, and response headers. Effectively capturing this semantic information is crucial for accurately identifying tracking requests.

• **Challenge 2: Leveraging the structural information of the graph.** The TDG graph has a complex structure, with multiple edges between the same source and target nodes, as well as self-loop edges. Effectively leveraging this structural information is essential for accurately classifying edges.

• **Challenge 3: Validating the effectiveness of the LLM-based labeling component.** To ensure that the LLM-based labeling component can accurately identify tracking requests, we need to design an effective validation method to evaluate its performance.

Therefore, we propose a model architecture that combines a pre-trained language model with a graph neural network to address these challenges. The overall architecture is illustrated in Figure 2. As shown in the figure, the architecture consists of four main components. *Data Collector* (C1) is responsible for collecting web traffic by simulating user browsing behavior. *Graph Constructor* (C2) constructs the TDG graph from the collected web traffic. *LLM-based labeling* (C3) employs a pre-trained language model to analyze semantic features derived from request and response headers as well as their origins, and to predict the tracking attribute of each edge. While this approach effectively annotates edge-level semantics, it faces fundamental limitations in modeling the full graph structure. Although Fatemi et al. [18] propose various encoding schemes to input graph structure into LLMs, applying such approaches to a dense graph with more than 20,000 nodes would either require partitioning the graph into multiple subgraphs—resulting in the loss of structural information—or using a context window of approximately 2–5 million tokens, which far exceeds the capabilities of current LLMs. Given these structural constraints and the inherent difficulty of modeling graph data with sequential transformers, we employ Graph Attention Networks (GAT) instead of LLMs for end-to-end detection, enabling effective learning over the complete graph topology. *Graph Learning* (C4) is the core of this architecture, which employs GAT to learn edge representations by combining the semantic information of nodes and edges with the structural information of the graph, and classifies each edge as tracking or not. In the final detection phase, *Edge Classification* (C5) applies the pre-trained GAT model to infer and classify unknown edges within the complete Tracking Directed Graph (TDG), thereby identifying potential tracking connections in the wild.

4.1.1 Data Collector. To automatically simulate real user data and collect network traffic at scale, we employed the OpenWPM framework [15] and developed scripts to drive the Firefox browser with simulated browsing behaviors. The scripts incorporate random scrolling and random mouse movement operations to more closely emulate real user interactions. The detailed configuration of the data collector is described in Appendix A.1.

4.1.2 Graph Constructor. The Graph Constructor component is designed to construct the TDG from the collected web traffic. The main tasks of this component include two sub-tasks: graph construction and feature extraction.

Graph Construction. The graph construction sub-task involves parsing the collected web traffic data to extract nodes and edges, and then constructing the TDG based on the defined structure. Each unique website encountered in the web traffic is represented as a node in the graph, while each request from one website to another is represented as a directed edge. The graph construction process also involves handling multiple edges between the same source and target nodes, as well as self-loop edges.

Feature Extraction. The feature extraction sub-task involves extracting relevant features for each node and edge in the TDG. For each node, we extract features from its domain name. For each edge, we extract features from its request headers, response headers, request body, and response body. We utilize a pre-trained language model to encode the natural language information of nodes and edges into vector representations, ensuring that semantically similar nodes and edges are mapped to similar vector representations. The formal details of feature extraction have been described in § 3.

4.1.3 LLM-based Labeling. This component is designed to emulate the labeling knowledge of human experts by predicting the tracking attribute of an edge $e_{(u,v,i)} \in E$ based on its contextual information $C = (H_{\text{Request}}, H_{\text{Response}}, R_{\text{Request}}, R_{\text{Response}})$. Specifically, the input to this component is the contextual information C of edge $e_{(u,v,i)}$, and the output is its tracking attribute $\mathcal{T}(e_{(u,v,i)}) \in \{0, 1\}$. To this end, we employ the large language model Qwen3 [49] as the underlying model M_λ for LLM-based labeling.

For the expert-labeled ground truth $D_{\text{truth}} = \{(c_i, y_i)\}_{i=1}^N$, our objective is to enable M_λ to assert each tracking context as accurately as possible while maintaining generalization capability. To this end, instead of directly feeding ground truth samples into the model we represent the sample features in natural language form and embed them into carefully designed prompts. Specifically, we construct a prompt template detailed in Appendix B that contains task descriptions along with both positive and negative sample features. The generated prompt is then provided to the large language model M_λ to obtain predictions $\hat{y}_i = M_\lambda(c_i)$. Finally, we adopt the F_1 -score as the primary evaluation metric to assess the performance of M_λ on the ground truth dataset. Experimental results in § 5.5 demonstrate that M_λ performs well (F_1 -score of 98.17%), thereby validating its effectiveness as the LLM-based labeling component.

4.1.4 Graph Learning. Given a constructed TDG along with its nodes and edges, our task is to predict the tracking nature of the edges based on the information from both nodes and edges.

Design Philosophy. To achieve this goal, there are at least three potential solutions. The first approach is to employ traditional machine learning methods, such as Random Forests [8], for edge classification. For example, AdGraph [24] applies Random Forests to classify advertising requests. However, this approach fails to fully leverage the structural information of the graph. The second approach is to classify requests directly based on their semantic features, such as URLs and request headers, as discussed by Karakaya et al. [26]. Nevertheless, this method overlooks the structural dependencies encoded in the graph. The third approach is to employ Graph Neural Network (GNN)-based classification, as exemplified by WTAGRAPH [50], which applies GNNs to classify advertising requests. However, the ground truth dataset used in WTAGRAPH is constructed from ad-filtering lists, whose accuracy and completeness are questionable due to their reliance on crowd-sourcing. Motivated by the strengths of the latter two approaches, we design a GNN-based edge classification component. This component leverages pretrained language models to extract semantic features for nodes and edges, while employing GNNs to capture the structural information of the graph, thereby combining both semantic and structural advantages.

This component is designed to learn the representations of each edge in the TDG graph and classify them based on their representations. The input to this component is the TDG graph $G = (V, E, X^V, X^E, \mathcal{T})$. Using the LLM-based labeling component M_λ described above, the tracking attribute of each edge $e_{(u,v,i)} \in E$ is predicted, yielding the edge label $\hat{y}_e = M_\lambda(c)$. The graph learning component then learns edge representations and performs classification by combining the edge feature vector x_e , the feature vectors of the source node x_u and target node x_v , as well as the structural information of the graph. Specifically, the graph learning component adopts GAT [47] as the backbone graph neural network, where multiple GAT layers are employed to learn node representations, which are further combined with edge feature vectors to learn edge representations.

To implement edge classification on the Tracking Directed Graph $G = (V, E, X^V, X^E, \mathcal{T})$, we design a TGNN architecture that incorporates both node and edge features into the graph attention mechanism as shown in Algorithm 1 in Appendix C.

4.1.5 Edge Classification. The final phase of TGNN is edge classification. In this stage, the pre-trained Graph Attention Network (GAT) model from the phase of Graph Learning is deployed to classify previously unlabeled edges in the Tracking Directed Graph (TDG) for real-world tracking detection. Our findings regarding pixel tracking in real-world environments (§ 5.4) are derived from a comprehensive analysis combining the labeled edges generated by the LLM-based Labeling component and the newly classified edges identified through the Edge Classification component.

5 Experiments

To evaluate the effectiveness of TGNN, we design our experiments to answer the following research questions. **RQ1:** How effective is TGNN in detecting web tracking requests? (§ 5.2) **RQ2:** What is the contribution of each component in TGNN to the overall performance? (§ 5.3) **RQ3:** What new tracking requests can TGNN discover in real-world scenarios? (§ 5.4) **RQ4:** Can the data labeling

component surpass filter lists and match human expert annotation capabilities? (§ 5.5)

5.1 Experimental Setting

Data Collection & Data Labeling. As explained in § 4.1, we implemented the data collector component of TGNN as a script that drives the Firefox browser using the OpenWPM framework [15]. The detailed description of data collection and data labeling can be found in Appendix A.1 and Appendix A.2, respectively. The detailed settings can be found in Appendix A.3.

Evaluation Metrics. We adopt the widely used metrics in classification tasks, including accuracy, precision, recall, and F_1 -score, to evaluate the performance of TGNN. We expect the harmonic mean of precision and recall, namely the F_1 -score, to achieve a high value, which indicates that the model attains strong overall performance.

Table 1: Effectiveness Evaluation of TGNN. The key machine learning techniques used are indicated in parentheses.

Method	Connected Requests				Isolated Requests			
	Accuracy		Precision		Recall		F_1 -score	
TGNN (LLM+GAT)	91.87%	89.61%	94.74%	87.91%	89.87%	81.32%	92.24%	84.49%
AdGraph (RF)	94.20%	95.13%	84.87%	11.58%	83.95%	17.46%	84.41%	13.93%
WebGraph (RF)	85.13%	61.63%	85.19%	66.17%	85.13%	61.63%	84.99%	63.29%
WTAGRAPH (GCN)	95.68%	95.81%	88.89%	17.98%	87.92%	26.67%	88.40%	21.48%

5.2 Effectiveness Evaluation (RQ1)

In this subsection, we address RQ1 through experimental evaluation. We partition the original data graph G into a training graph G_{train} and a testing graph G_{test} , ensuring $G = G_{train} \cup G_{test}$ and $G_{train} \cap G_{test} = \emptyset$, to assess the effectiveness of TGNN in detecting web tracking requests. We compare TGNN with AdGraph [24], WebGraph [39] and WTAGRAPH [50], all of which employ machine learning techniques for detecting web tracking requests. AdGraph and WebGraph utilizes a random forest classifier, while WTAGRAPH adopts a GCN model. Following the classification scheme in WTAGRAPH, we define *connected requests* (or *inter-graph requests*) as those associated with at least one edge between G_{train} and G_{test} , i.e., $\exists e(u,v,i) \in G, u \in G_{train}, v \in G_{test}$ or $v \in G_{train}, u \in G_{test}$, and *isolated requests* as those without such cross-graph edges, i.e., $\nexists e(u,v,i) \in G, u \in G_{train}, v \in G_{test}$ or $v \in G_{train}, u \in G_{test}$. More details settings of the experiments can be found in Appendix A.3.

As shown in Table 1, TGNN outperforms AdGraph and WTAGRAPH in terms of F_1 -score, but is slightly inferior in terms of accuracy. This is because AdGraph and WTAGRAPH do not perform data balancing, causing the models to be biased toward predicting non-tracking requests, thereby increasing accuracy but reducing F_1 -score, as also noted in the Limitations section of the WTAGRAPH study [50]. In contrast, TGNN incorporates undersampling during training to balance the number of positive and negative samples, which leads to better performance in terms of F_1 -score. For isolated requests, TGNN also surpasses AdGraph and WTAGRAPH, demonstrating its ability to effectively detect web tracking requests under the inductive learning setting. Moreover, discarding the edges connecting the training and testing graphs results in a significant

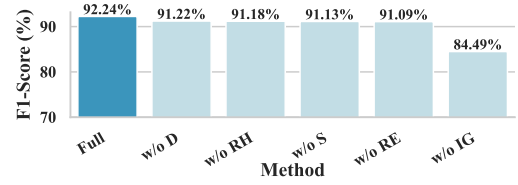


Figure 3: Ablation Analysis of TGNN. "w/o IG" means without inter-graph edges. "w/o D" means without destination URL features. "w/o S" means without source domain features. "w/o RH" means without request header features. "w/o RE" means without response header features.

performance drop, indicating that these edges play an important role in capturing the relationship between training and test data.

5.3 Ablation Analysis (RQ2)

In this subsection, we address RQ2 through ablation experiments. We design the following ablation studies to evaluate the contribution of each component in TGNN to the overall performance: 1. An experiment removing inter-graph edges; 2. An experiment removing destination URL features from edges; 3. An experiment removing source domain features from edges; 4. An experiment removing request header features from edges; 5. An experiment removing response header features from edges.

The results of the ablation experiments are presented in Figure 3. From the results, we observe that removing inter-graph edges leads to a significant drop in performance, indicating their crucial role in capturing relationships between training and testing data. Additionally, removing destination URL features results in a notable decrease in precision and recall, highlighting their importance in identifying tracking requests. The removal of source domain features also negatively impacts performance, albeit to a lesser extent. Furthermore, eliminating request and response header features causes a decline in all metrics, underscoring their contribution to the model's effectiveness. Overall, these findings demonstrate that each component of TGNN plays a vital role in achieving optimal performance in detecting web tracking requests.

5.4 Findings in the Wild (RQ3)

In this subsection, we analyze the results produced by TGNN to address RQ3. Our analysis uncovers several concerning findings. To classify the websites, we utilized the dataset provided by Homepage2Vec [31], which categorizes websites into 14 classes, including *news*, *shopping*, and *kids & teens*, among others. Based on this classification dataset, we categorized the websites and calculated the number of tracking and non-tracking requests within each category. Figure 4 illustrates the distribution of tracking and non-tracking requests across different website categories. As shown in the figure, websites in the *arts*, *recreation*, and *sports* categories exhibit relatively high proportions of tracking requests, accounting for 54.7%, 54.5%, and 48.0%, respectively. These websites typically embed multiple third-party services, such as Magnite and Google Analytics, which collect user browsing behavior data and conduct tracking.

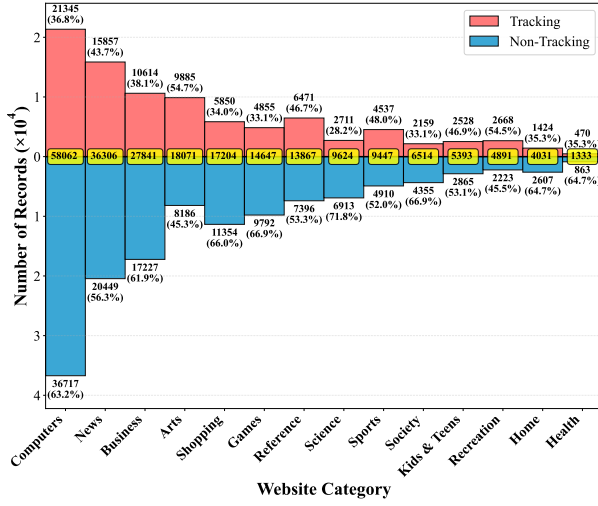


Figure 4: The distribution of tracking behaviors across 14 categories of websites.

Among a total of 805,651 traffic records, approximately 1.87% of the traffic employed Google Analytics for tracking, while about 1.12% employed Magnite. Furthermore, 3,297 (14.19%) distinct domains interacted with Google Analytics, and 592 (2.55%) distinct domains interacted with Magnite. These findings indicate that such websites may leverage third-party services to conduct cross-domain and persistent tracking of user browsing behaviors, thereby infringing upon user privacy. As shown in Figure 6 in Appendix D, we visualize the top 2 tracking services with the highest in-degrees and their connected domains.

To further investigate the extent to which existing websites track vulnerable groups, such as children and teenagers, we conducted a focused analysis with the assistance of human experts. Following the methodology proposed by Moti et al. [33], we utilized the list of child-oriented websites compiled by Kaspersky [27] and combined it with expert annotations to identify nine websites targeting children and teenagers. These websites primarily fall into categories such as gaming and education. We analyzed 357 traffic records collected from these nine websites and found that 167 (46.78%) of them contained tracking requests. This proportion is consistent with the tracking request ratio (46.9%) observed for the *kids & teens* category in Figure 4. Most of these tracking requests originate from advertising services such as Google Analytics, which collect users' browsing behavior data and pose a significant threat to children's privacy and online safety.

5.5 Evaluation of Labeling Component (RQ4)

In this subsection, we address RQ4 by evaluating the performance of the data labeling component in comparison to filter lists and human expert annotations. We conduct experiments to assess whether the data labeling component can surpass filter lists and match the annotation capabilities of human experts.

We use the dataset annotated by human experts as the ground truth to evaluate the performance of the data labeling component.

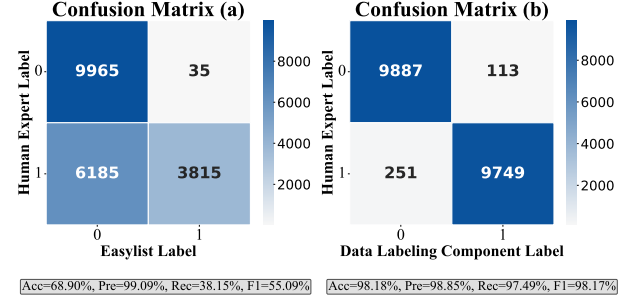


Figure 5: Confusion Matrix(a) demonstrates the performance of EasyList against human expert annotations. Confusion Matrix(b) demonstrates the performance of the data labeling component against human expert annotations.

To further compare the performance of the data labeling component with filter lists, we adopt a widely used filter list: EasyList [44]. We apply this filter list to the ground truth dataset and compute their accuracy, precision, recall, and F_1 -score. The Python library `adblockparser` [43] was used to parse the filter lists and apply them to the dataset. Figure 5(a) shows the confusion matrix of EasyList against human expert annotations, and Figure 5(b) shows the confusion matrix of the data labeling component against human expert annotations. From the confusion matrices, we observe that EasyList has a lower F_1 -score compared to the data labeling component, indicating that the data labeling component can better match human expert annotations. Specifically, EasyList tends to miss a significant number of tracking requests (high false negatives), while the data labeling component demonstrates a more balanced performance with fewer false negatives and false positives. This suggests that the data labeling component is more effective in identifying tracking requests in line with human expert judgments.

As shown in Figure 5, the F_1 -score of the data labeling component reaches 98.17%, which is slightly higher than that of TGNN (92.24%). In our experiment comparing the data labeling component with filter lists, human experts tended to select and label data that lie far from the decision boundary, whereas TGNN was trained and validated on the complete dataset. This difference explains why TGNN's F_1 -score is slightly lower than that of the data labeling component. Within the data labeling component, the LLM serves as a teacher model that distills tracking-related knowledge by integrating its own representations with expert-provided cues, thereby transferring this knowledge to TGNN and making its predictions more consistent with expert annotations than those derived from filter lists. As discussed in § 4.1, due to the inherent limitations of LLMs, it is infeasible to encode the full graph structure into their context. To address this, we innovatively integrate the LLM as a component within TGNN, thereby enhancing TGNN's performance.

6 Discussion

Our Goals. Some real-world websites include traffic designed to coerce users into disabling filter lists (e.g., by stopping or degrading content when they are enabled). Although whitelisting can serve as a short-term compromise, its effectiveness in protecting user

privacy is limited. Our study focuses on detecting, rather than blocking, advertising requests, aiming to strengthen privacy protection. By identifying tracking behaviors, we can provide users with more informed choices about their online activities and enhance their overall privacy.

Findings and Implications. Our study reveals that TGNN can effectively detect web tracking behaviors by leveraging the structural information of web traffic data. The LLM-based labeling component, which encapsulates human expert knowledge, significantly enhances the model's performance. This suggests that combining domain expertise with advanced machine learning techniques can lead to more robust and accurate detection systems. The implications of our findings extend to improving user privacy and security on the web, as more effective tracking detection can help mitigate unwanted tracking activities.

Limitations and Potential Future Work. Although we innovatively designed and employed the LLM-based labeling component to extend human expert knowledge, and verified both its effectiveness as an extension of expert knowledge and the effectiveness of the TGNN design, our work still has certain limitations. For instance, our approach cannot process traffic data in real time. AdGraph [24] achieved real-time detection by leveraging a modified Chromium kernel. Future work can incorporate the ideas of AdGraph to design a real-time TGNN-based detection system. In addition, our work only utilized features such as request URLs, request headers, origin domains, and response headers. Future work may integrate additional features, such as request bodies and DOM features, to further enhance detection performance.

7 Related Work

The stealthiness of pixel tracking has attracted widespread attention. Ruohonen et al. [36] further confirmed the pervasive presence of invisible pixels and demonstrated that they can be classified with high accuracy (95.6%) using machine learning techniques, while rule-based ad-blocking tools fail to completely prevent such tracking. Hu et al. [22] reported that third-party tracking is highly prevalent in email, where trackers employ various obfuscation strategies to conceal their presence, including but not limited to falsifying declared image sizes or using redirection to hide trackers. These findings indicate that pixel tracking is a common and effective method for monitoring users, and its stealthy nature poses significant challenges for detection and prevention. Existing methods for detecting pixel tracking can be broadly categorized into the following two paradigms.

Rule-based tracking detection. Identify tracking behaviors by inspecting code or traffic against a predefined set of rules. Representative examples include Adblock Plus [42] and uBlock Origin [21], which rely on rule lists such as EasyList [44] and EasyPrivacy [45]. The advantages of this approach lie in its simplicity and efficiency; however, its effectiveness is limited to previously defined rules and thus struggles to detect emerging tracking techniques. Moreover, due to the transparency of these rules, trackers can deliberately craft evasion strategies to bypass them and continue monitoring users. In addition, many rule lists are generated through crowdsourcing, resulting in inconsistent quality and increasing the likelihood of false positives and false negatives.

ML-based tracking detection. By extracting features from traffic [19, 38], various machine learning classifiers have been developed to detect trackers. Researchers have also shown the effectiveness of different machine learning classifiers on detecting trackers by using syntactic and semantic features of JavaScript code [23, 25, 48]. Recently, AdGraph [24], WebGraph [39] and WTAGRAPH [50] have been proposed to detect web trackers. The former approach modifies the browser kernel to transform the browser into a traffic collector, followed by traffic analysis using a random forest model. The latter leverages OpenWPM [15] to capture traffic and employs a graph neural network (GNNs), where websites are represented as graph nodes and traffic flows as edges, to perform edge classification. Although AdGraph and WTAGRAPH have explored this direction using machine learning techniques, their ground truth relies on crowdsourced filter lists. In § 5.5, we present experimental results showing that while such filter lists can block certain trackers, a significant portion remains undetected. Establishing a reliable ground truth is therefore fundamental to this study, which motivates our use of human expert knowledge to construct the ground truth dataset.

8 Conclusion

In this work, we empirically analyzed the prevalence and stealthiness of invisible tracking in the domain of privacy protection and proposed TGNN, a detection framework specifically designed to match the fundamental characteristics of pixel tracking. TGNN incorporates an LLM-driven data annotation module that acquires knowledge of pixel tracking patterns from a manually annotated dataset, thereby significantly expanding high-quality labeled data. Simultaneously, the GAT-based tracking classifier learns to aggregate both semantic features from HTTP traffic and structural representations of tracking requests across websites, enabling effective preservation of discriminative tracking characteristics. Experimental results show that our LLM-based labeling component not only surpasses conventional filter list-based methods by a large margin but also reaches an annotation quality on par with human experts. We conducted a large-scale analysis of tracking behaviors across the Alexa top-10K websites and found that third-party tracking pixels are most prevalent in the arts, recreation, and sports websites. More critically, our study highlights the particular impact of invisible tracking on vulnerable populations: nearly half of all requests on child-oriented websites exhibit tracking behavior, underscoring potential threats to privacy and security.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No.92467201), the Youth Innovation Promotion Association CAS (Grant No.Y2023047), the Key Laboratory of Network Assessment Technology at Chinese Academy of Sciences and Beijing Key Laboratory of Network Security and Protection Technology.

References

- [1] Vibhor Agarwal, Yash Vekaria, Pushkal Agarwal, Sangeeta Mahapatra, Shounak Set, Sakthi Balan Muthiah, Nishanth Sastry, and Nicolas Kourtellis. 2021. Under the Spotlight: Web Tracking in Indian Partisan News Websites. In *Proceedings of*

- the International AAAI Conference on Web and Social Media* (2021-05-22), Vol. 15. 26–37. doi:10.1609/icwsm.v15i1.18038
- [2] Gonalo Almeida Teixeira, Miguel Mira da Silva, and Ruben Pereira. 2019. The Critical Success Factors of GDPR Implementation: A Systematic Literature Review. 21, 4 (2019), 402–418. doi:10.1108/DPRG-01-2019-0007
- [3] Meenatchi Sundaram Muthu Selva Annamalai, Igor Bilogrevic, and Emiliano De Cristofaro. 2024. FP-fed: Privacy-Preserving Federated Detection of Browser Fingerprinting. In *Proceedings 2024 Network and Distributed System Security Symposium* (San Diego, CA, USA, 2024). Internet Society. doi:10.14722/ndss.2024.24360
- [4] Meenatchi Sundaram Muthu Selva Annamalai, Emiliano De Cristofaro, and Igor Bilogrevic. 2025. Beyond the Crawl: Unmasking Browser Fingerprinting in Real User Interactions. In *Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025– 2 May 2025* (2025), Guodong Long, Michale Blumstein, Yi Chang, Liane Lewin-Eytan, Zi Helen Huang, and Elad Yom-Tov (Eds.). ACM, 3896–3907. doi:10.1145/3696410.3714871
- [5] Muhammad Ahmad Bashir and Christo Wilson. 2018. Diffusion of User Tracking Data in the Online Advertising Ecosystem. 2018, 4 (2018), 85–103. doi:10.1515/popets-2018-0033
- [6] Paschalis Bekos, Panagiotis Papadopoulos, Evangelos P. Markatos, and Nicolas Kourtellis. 2023. The Hitchhiker’s Guide to Facebook Web Tracking with Invisible Pixels and Click IDs. In *Proceedings of the ACM Web Conference 2023* (Austin TX USA, 2023-04-30) (WWW ’23). ACM, 2132–2143. doi:10.1145/3543507.3583311
- [7] Dino Bollinger, Karel Kubicek, Carlos Cotrini, and David Basin. 2022. Automating Cookie Consent and {GDPR} Violation Detection. 2893–2910. <https://www.usenix.org/conference/usenixsecurity22/presentation/bollinger>
- [8] Leo Breiman. 2001. Random Forests. 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324
- [9] Tomasz Bujlow, Valentin Carrel-Espanol, Beom-Ryeol Lee, and Pere Barlet-Ros. 2017. A Survey on Web Tracking: Mechanisms, Implications, and Defenses. 105, 8 (2017), 1476–1510. doi:10.1109/JPROC.2016.2637878
- [10] Ismael Castell-Uroz, Josep Solé-Pareta, and Pere Barlet-Ros. 2021. TrackSign: Guided Web Tracking Discovery. In *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications* (Vancouver, BC, Canada, 2021-05-10). IEEE, 1–10. doi:10.1109/INFOCOM42981.2021.9488842
- [11] Ismael Castell-Uroz, Josep Solé-Pareta, and Pere Barlet-Ros. 2020. Network Measurements for Web Tracking Analysis and Detection: A Tutorial. 23, 9 (2020), 50–57. doi:10.1109/MIM.2020.9289071
- [12] Apple Developer. 2025. *User Privacy and Data Use*. <https://developer.apple.com/app-store/user-privacy-and-data-use>
- [13] Digital Markets Act (Dma). 2025. *The Digital Markets Act Is the EU’s Law to Make the Markets in the Digital Sector Fairer and More Contestable*. https://digital-markets-act.ec.europa.eu/index_en
- [14] Asmaa El Fraihi, Nardjes Amieur, Walter Rudametkin, and Oana Goga. 2024. Client-Side and Server-Side Tracking on Meta: Effectiveness and Accuracy. 2024, 3 (2024), 431–445. doi:10.56553/popets-2024-0086
- [15] Steven Englehardt and Arvind Narayanan. 2016. Online Tracking: A 1-Million-Site Measurement and Analysis. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna Austria, 2016-10-24). ACM, 1388–1401. doi:10.1145/2976749.2978313
- [16] Council of the European Union European Parliament. 2016. *Regulation (Eu) 2016/679 of the European Parliament and of the Council*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>
- [17] Aurore Fass, Michael Backes, and Ben Stock. 2019. JStap: A Static Pre-Filter for Malicious JavaScript Detection. In *Proceedings of the 35th Annual Computer Security Applications Conference* (San Juan Puerto Rico USA, 2019-12-09). ACM, 257–269. doi:10.1145/3359789.3359813
- [18] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. <https://openreview.net/forum?id=IuXR1CCrSi>
- [19] David Gugelmann, Markus Happe, Bernhard Ager, and Vincent Lenders. 2015. An Automated Approach for Complementing Ad Blockers’ Blacklists. 2015, 2 (2015), 282–298. doi:10.1515/popets-2015-0018
- [20] Aniko Hannak, Gary Soeller, David Lazer, Alan Mislove, and Christo Wilson. 2014. Measuring Price Discrimination and Steering on E-commerce Web Sites. In *Proceedings of the 2014 Conference on Internet Measurement Conference* (New York, NY, USA, 2014-11-05) (IMC ’14). Association for Computing Machinery, 305–318. doi:10.1145/2663716.2663744
- [21] Raymond Hill. 2025. *uBlock Origin*. <https://github.com/gorhill/uBlock>
- [22] Hang Hu, Peng Peng, and Gang Wang. 2019. Characterizing Pixel Tracking through the Lens of Disposable Email Services. In *2019 IEEE Symposium on Security and Privacy (SP)* (San Francisco, CA, USA, 2019-05). IEEE, 365–379. doi:10.1109/SP.2019.00033
- [23] Muhammad Ikram, Hassan Jameel Asghar, Mohamed Ali Kaafar, Anirban Mahanti, and Balachandhar Krishnamurthy. 2017. Towards Seamless Tracking-Free Web: Improved Detection of Trackers via One-Class Learning. 1 (2017), 79–99.
- [24] Umar Iqbal, Peter Snyder, Shitong Zhu, Benjamin Livshits, Zhiyun Qian, and Zubair Shafiq. 2020. AdGraph: A Graph-Based Approach to Ad and Tracker Blocking. In *2020 IEEE Symposium on Security and Privacy (SP)* (San Francisco, CA, USA, 2020-05). IEEE, 763–776. doi:10.1109/SP40000.2020.00005
- [25] Andrew J. Kaizer and Minaxi Gupta. 2016. Towards Automatic Identification of JavaScript-oriented Machine-Based Tracking. In *Proceedings of the 2016 ACM on International Workshop on Security and Privacy Analytics* (New York, NY, USA, 2016-03-11) (IWSPA ’16). Association for Computing Machinery, 33–40. doi:10.1145/2875475.2875479
- [26] Can Karakaya, Hakan Toguc, Ridvan Salih Kuzu, and Ali Hakan Buyuklu. 2018. Survey of Cross Device Matching Approaches with a Case Study on a Novel Database. In *2018 3rd International Conference on Computer Science and Engineering (UBMK)* (Sarajevo, 2018-09). IEEE, 139–144. doi:10.1109/UBMK.2018.8566307
- [27] Kaspersky. 2025. *The List of Kids’ Websites*. <https://kids.kaspersky.com/kids-website-list/>
- [28] Peter Königs. 2022. Government Surveillance, Privacy, and Legitimacy. 35, 1 (2022), 8. doi:10.1007/s13347-022-00503-9
- [29] Hieu Le, Salma Elmalaki, Athina Markopoulou, and Zubair Shafiq. 2023. AutoFR: Automated Filter Rule Generation for Adblocking. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023* (2023), Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, 7535–7552. <https://www.usenix.org/conference/usenixsecurity23/presentation/le>
- [30] Changmin Lee and Soeul Son. 2023. Adcpg: Classifying Javascript Code Property Graphs with Explanations for Ad and Tracker Blocking. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (2023), 3505–3518. doi:10.1145/3576915.3623084
- [31] Sylvain Lugeon, Tiziano Piccardi, and Robert West. 2022. Homepage2Vec: Language-Agnostic Website Embedding and Classification. In *Proceedings of the International AAAI Conference on Web and Social Media* (2022-05-31), Vol. 16. 1285–1291. doi:10.1609/icwsm.v16i1.19380
- [32] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. 2013. Rectifier Nonlinearities Improve Neural Network Acoustic Models. In *Proc. Icml* (2013), Vol. 30. Atlanta, GA, 3.
- [33] Zahra Moti, Asuman Senol, Hamid Bostani, Frederik Zuiderveen Borgesius, Vee-lasha Moonsamy, Arunesh Mathur, and Gunes Acar. 2024. Targeted and Troublesome: Tracking and Advertising on Children’s Websites. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024-05). 1517–1535. doi:10.1109/SP54263.2024.00118
- [34] Shaorun Munir, Sandra Siby, Umar Iqbal, Steven Englehardt, Zubair Shafiq, and Carmela Troncoso. 2023. CookieGraph: Understanding and Detecting First-Party Tracking Cookies. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (2023), 3490–3504. doi:10.1145/3576915.3616586
- [35] Franziska Roesner, Tadayoshi Kohno, and David Wetherall. 2012. Detecting and Defending against Third-Party Tracking on the Web. In *Symposium on Networked Systems Design and Implementation* (San Jose, CA and USA, 2012-04-25) (NSDI’12). USENIX Association, 155–168. <https://www.semanticscholar.org/paper/96df51029b8f6c936c9909bd45a9b08498d1ffa>
- [36] Jukka Ruohonen and Ville Leppänen. 2018. Invisible Pixels Are Dead, Long Live Invisible Pixels!. In *Proceedings of the 2018 Workshop on Privacy in the Electronic Society* (Toronto Canada, 2018-01-15). ACM, 28–32. doi:10.1145/3267323.3268950
- [37] Iskander Sanchez-Rola, Matteo Dell’Amico, Platon Kotzias, Davide Balzarotti, Leyla Bilge, Pierre-Antoine Vervier, and Igor Santos. 2019. Can I Opt out yet? GDPR and the Global Illusion of Cookie Control. In *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security* (New York, NY, USA, 2019-07-02) (Asia CCS ’19). Association for Computing Machinery, 340–351. doi:10.1145/3321705.3329806
- [38] Anastasia Shuba, Athina Markopoulou, and Zubair Shafiq. 2018. NoMoAds: Effective and efficient cross-app mobile ad-blocking. (2018). doi:10.1515/popets-2018-0035
- [39] Sandra Siby, Umar Iqbal, Steven Englehardt, Zubair Shafiq, and Carmela Troncoso. 2022. {WebGraph}: Capturing Advertising and Tracking Information Flows for Robust Blocking. 2875–2892. <https://www.usenix.org/conference/usenixsecurity22/presentation/siby>
- [40] Kyungmin Sim, Honyeong Heo, and Haehyun Cho. 2024. Combating Web Tracking: Analyzing Web Tracking Technologies for User Privacy. 16, 10 (2024), 363. doi:10.3390/ft16100363
- [41] Junhua Su and Alexandros Kapravelos. 2023. Automatic Discovery of Emerging Browser Fingerprinting Techniques. In *Proceedings of the ACM Web Conference 2023* (Austin TX USA, 2023-04-30). ACM, 2178–2188. doi:10.1145/3543507.3583333
- [42] The Adblock Plus authors. 2025. *Adblock Plus*. <https://adblockplus.org/en/>
- [43] The adblockparser authors. 2025. *Adblockparser: Parser for Adblock plus Rules*. <https://github.com/scrapinghub/adblockparser>
- [44] The EasyList authors. 2025. *EasyList*. <https://easylist.to/easylist/easylist.txt>
- [45] The EasyPrivacy authors. 2025. *EasyPrivacy*. <https://easylist.to/easylist/easyprivacy.txt>
- [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Neural Information Processing Systems* (2017-06-12). arXiv, 5998–6008. doi:10.48550/arXiv.1706.03762
- [47] Petar Velićković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. <https://openreview>.

- net/forum?id=rJXmpikCZ
- [48] Qianru Wu, Qixu Liu, Yuqing Zhang, Peng Liu, and Guanxing Wen. 2016. A Machine Learning Approach for Detecting Third-Party Trackers on the Web. In *Computer Security – ESORICS 2016* (Cham, 2016), Ioannis Askoxylakis, Sotiris Ioannidis, Sokratis Katsikas, and Catherine Meadows (Eds.). Springer International Publishing, 238–258. doi:10.1007/978-3-319-45744-4_12
 - [49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruizhe Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. *Qwen3 Technical Report*. arXiv:2505.09388 [cs] doi:10.48550/arXiv.2505.09388
 - [50] Zhiyu Yang, Weiping Pei, Monchu Chen, and Chuan Yue. 2022. WTAGRAPH: Web Tracking and Advertising Detection Using Graph Neural Networks. In *2022 IEEE Symposium on Security and Privacy (SP)* (San Francisco, CA, USA, 2022-05). IEEE, 1540–1557. doi:10.1109/SP46214.2022.9833670

A Details of Experiments Settings

A.1 Data Collection

We implemented the data collector component of TGNN as a script that drives the Firefox browser using the OpenWPM framework [15]. The script simulates user browsing behavior by incorporating random scrolling and mouse movement operations. We executed this script on a PC with an Intel i9-13900HX CPU, 32GB RAM, and a 1TB SSD. During the procedure of data collection, we capture the request/response origins and headers of HTTP traffic from Alexa top-10K websites. Due to the discontinuation of Alexa’s services in 2022, we sourced the archived data from <https://www.expireddomains.net/alexa-top-websites/>.

In our data collection phase, we used Firefox v130.0.1 and ran 20 concurrent browser instances with a 10-minute per-site browsing time cap; additionally, we employed proxies to fix the IP address for improved reproducibility. We selected the Alexa top-10K websites as our crawl set and executed the aforementioned simulated user browsing scripts on each site. Using this setup, we collected a large volume of network traffic data that served as a rich resource for subsequent graph construction and analysis.

A.2 Data Labeling

We leverage human experts and heuristic rules to annotate the data as ground truth. By analyzing the traffic data collected from 10,000 websites, we observed a large number of image requests of size 1×1 , as well as 1×2 or 2×1 image requests used to evade ad blockers. These images contain no actual content, but their request headers or lines carry substantial tracking information. We label such image requests as potential tracking requests. Subsequently, we apply heuristic rules to inspect the request lines and headers, where the rules include checking keys in the request headers and parameters in the request lines. Finally, human experts validate the ground truth dataset. The resulting ground truth dataset contains 20,000 requests, among which 10,000 are labeled as tracking requests and 10,000 as non-tracking requests. This dataset ensures high-quality annotations and serves to validate the effectiveness of our proposed method. The specific model of the large language model used in this paper is Qwen3-Plus.

A.3 Experiment Settings

The dataset generated by the data labeling component is split into training and testing sets with a ratio of 80% and 20%, respectively. We adopt accuracy, precision, recall, and F_1 -score as evaluation metrics. To further validate the performance of TGNN under the inductive learning setting, we also design an experiment where edges connecting the training and testing graphs are discarded. The results are presented in Table 1. We implement TGNN using the PyTorch framework and conduct training and evaluation on an NVIDIA V100 GPU.

For TGNN hyperparameters, we set the number of GAT hidden layers to 2, the hidden dimension to 16, and the number of attention heads to 14. We apply dropout with a rate of 0.5. The model is trained for 1000 epochs using the Adam optimizer with a learning rate of 0.005.

B Prompts

In this part, we show the prompts designed for annotations. We require LLMs to output "Yes" or "No" for each line of input, where "Yes" indicates that the traffic is intended for tracking, and "No" indicates that it is not.

Prompt for Data Labeling Component

You are an assistant skilled in traffic analysis. Each line below provides a data entry, and each entry is a quadruple consisting of the request URL, the referrer URL, the request body, and the response body. You need to analyze each of the quadruple separately and determine whether the traffic is intended for tracking.

You should consider the following factors: 1. Whether the request URL contains sensitive information that can be used to track users. 2. Whether the request URL belongs to a site used for tracking or providing tracking services. 3. Whether, based on the referrer URL, the request URL is used for tracking or providing tracking services. 4. Whether the request body and response body contain sensitive information that can be used to track users.

Sensitive information that can be used for tracking includes, but is not limited to, display resolution, IP address, MAC address, device ID, advertising tracking ID, device fingerprinting information, etc.

After your analysis, if the traffic contains sensitive information used for tracking, respond with "Yes"; otherwise, respond with "No". Your response must not include any analysis or additional information, but only "Yes", "No", and line breaks. Do not include any other characters under any circumstances. Answer with "Yes" or "No".

{QUADRUPLES DATA}

C Key Implementation Details of TGNN

As shown in Algorithm 1. First, node and edge feature matrices X^V and X^E are initialized. For each layer $l \in \{1, \dots, L\}$ and each attention head $h \in \{1, \dots, H\}$, the attention coefficients $\alpha_{ij}^{(h)}$ are computed by concatenating the features of the source node, target node, and the corresponding edge, followed by a LeakyReLU [32]

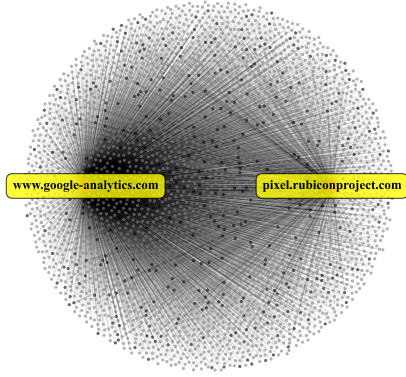


Figure 6: Top 2 in-degree tracking services and their connected domains.

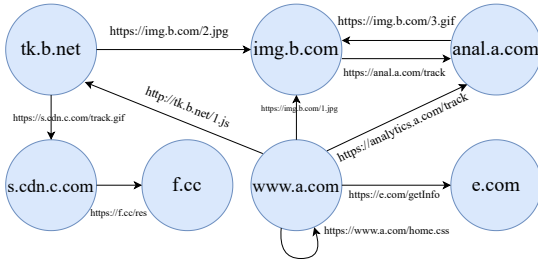


Figure 7: An example of Tracking Directed Graph (TDG) constructed from web traffic. Each node represents a website, and each directed edge symbolizes a request between sites.

activation and a Softmax normalization. The updated node representation $\mathbf{h}_i^{(h)}$ is then obtained by aggregating the weighted features of its neighbors. The outputs of all attention heads are combined and further refined through batch normalization, ELU activation, and dropout regularization.

After L layers of message passing, the final node representations $\mathbf{H}^{(L)}$ are obtained. For edge classification, the representation of each edge e_{ij} is constructed by concatenating the final embeddings of its incident nodes with the original edge features, i.e., $\mathbf{z}_e = \phi(\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{x}_e)$. This joint representation is then fed into a classification layer with parameters $(\mathbf{W}_{\text{class}}, \mathbf{b}_{\text{class}})$, yielding the predicted edge label $\hat{y}_e \in \{0, 1\}$.

Algorithm 1 Key Implementation Details of TGNN.

Require: Tracking Directed Graph $G = (V, E, X^V, X^E, \mathcal{T})$

Ensure: Predicted edge labels $\hat{y}_e \in \{0, 1\}$ for all $e \in E$

```

1: Initialize node features  $X^V$  and edge features  $X^E$ 
2:  $L \leftarrow$  number of layers,  $H \leftarrow$  number of attention heads
3: function GRAPHATTENTIONWITHEDGEFEATS( $g, \mathbf{H}^{(l-1)}, X^E$ )
4:   for each attention head  $h$  do
5:      $\alpha_{ij}^{(h)} \leftarrow \text{Softmax} \left( \text{LeakyReLU} \left( \mathbf{a}^{(h)\top} [\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{x}_e] \right) \right)$ 
6:      $\mathbf{h}_i^{(h)} \leftarrow \sigma \left( \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(h)} \mathbf{W}^{(h)} \mathbf{h}_j \right)$ 
7:   end for
8:    $\mathbf{H}^{(l)} \leftarrow \text{CombineHeads}(\mathbf{h}^{(1)}, \dots, \mathbf{h}^{(H)})$ 
9:   Apply ELU and Dropout to  $\mathbf{H}^{(l)}$ 
10:  return  $\mathbf{H}^{(l)}$ 
11: end function
12: function FORWARDPASS( $g, X^V, X^E$ )
13:   $\mathbf{H}^{(0)} \leftarrow X^V$ 
14:  for  $l \leftarrow 1$  to  $L$  do
15:     $\mathbf{H}^{(l)} \leftarrow \text{GraphAttentionWithEdgeFeats}(g, \mathbf{H}^{(l-1)}, X^E)$ 
16:  end for
17:  return  $\mathbf{H}^{(L)}$ 
18: end function
19: function EDGECLASSIFICATION( $g, \mathbf{H}, X^E$ )
20:  for each edge  $e_{ij} \in E$  do
21:     $\mathbf{z}_e \leftarrow \phi(\mathbf{h}_i \parallel \mathbf{h}_j \parallel \mathbf{x}_e)$ 
22:     $\hat{y}_e \leftarrow \sigma(\mathbf{W}_{\text{class}} \mathbf{z}_e + \mathbf{b}_{\text{class}})$ 
23:  end for
24:  return  $\hat{\mathbf{Y}}$ 
25: end function
26:  $\mathbf{H}^{(L)} \leftarrow \text{ForwardPass}(G, X^V, X^E)$ 
27:  $\hat{\mathbf{Y}} \leftarrow \text{EdgeClassification}(G, \mathbf{H}^{(L)}, X^E)$ 

```

D Visualization

Figure 6 demonstrates the top 2 tracking services with the highest in-degrees and their connected domains, where the red nodes represent the top 2 tracking services, while the grey and black nodes represent the domains connected to one or both of these services, respectively.

Figure 7 illustrates an example of a TDG constructed from web traffic. Each node in the graph represents a website, and each directed edge represents a request from one website to another. Due to the characteristics of web traffic, multiple edges may exist between the same source and target nodes, each corresponding to a different request. To uniquely identify each edge, we denote it as a triplet $e_{(u,v,i)}$. Self-loop edges, where a node issues a request to itself (i.e., $u = v$), are also permitted in this setting.